

RISC_AXI UART Bootloader Project

Custom RV32I SoC with AXI-Lite/APB, AXI-Stream DMA, and RAM-loaded C firmware

Table of Contents

01 Project Goal - Little MCU Design

02 Reference & Direction - STM32F103 Analysis & Redesign

03 System Architecture - SocTop, Bus Fabric, Memory Map

04 Firmware Flow - UART Bootloader, Linker, RAM App

05 Core & Subsystems - Bus, Pipeline, Peripherals, PLIC

06 Application Scenario - Bootloader + RGB DMA image loop

07 Verification & Lessons - Waveforms, demo, lessons learned

1. Project Goal: Little MCU Design

직접 만든 RV32I core 위에서 C firmware를 빠르게 올려 실행하는 작은 MCU SoC

Little MCU 구성

CPU 하나만이 아니라 MCU처럼 동작하는 최소 시스템을 구성합니다.

- Custom RV32I Core
- Boot ROM / executable SRAM
- Memory-mapped peripheral

Peripheral 제어

CPU가 bus를 통해 GPIO, UART, SPI, Timer, PLIC 같은 주변장치를 제어합니다.

- AXI-Lite / APB 기반 register access
- Interrupt 기반 event 처리
- RGB stream demo로 확장성 확인

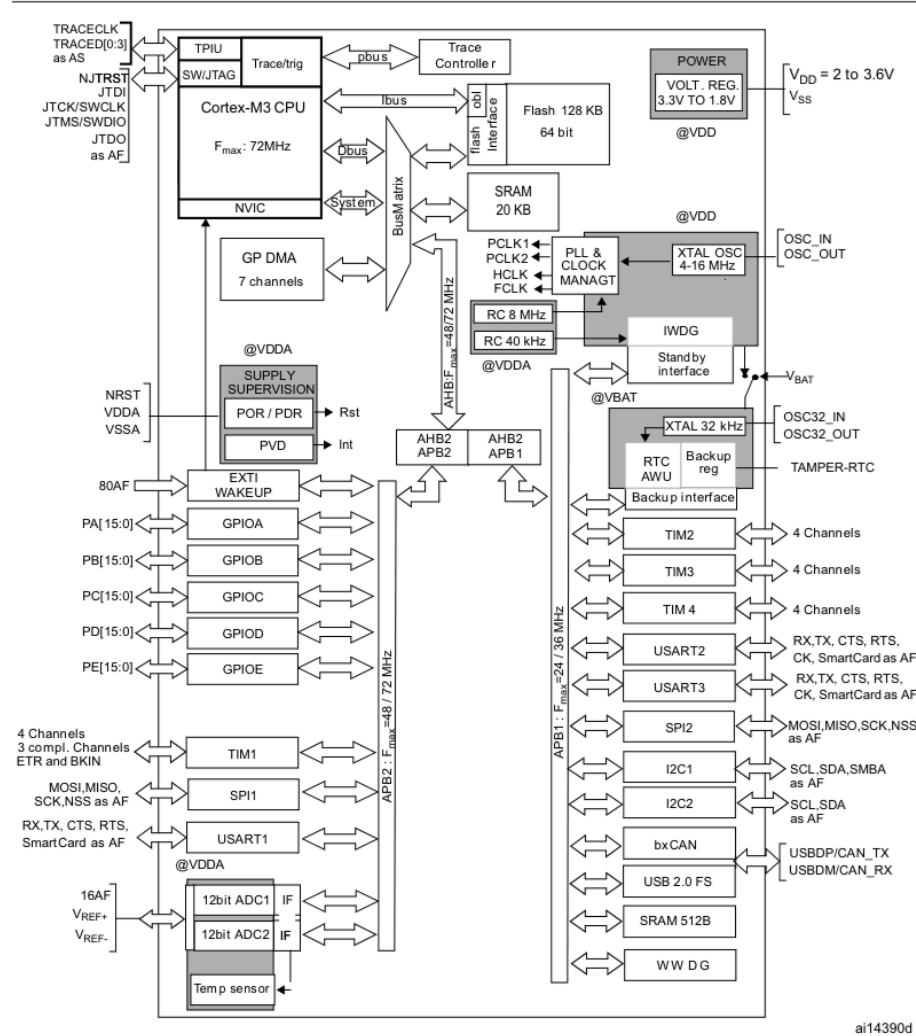
UART Bootloader

ROM에는 loader만 고정하고, PC에서 빌드한 C RAM app을 UART packet으로 SRAM에 올려 실행합니다.

- Bitstream 재생성 없는 firmware 반복
- RAXI packet validation
- RAM entry 0x20001000 jump

2. STM32F103 Reference & Redesign Direction

Figure 1. STM32F103xx performance line block diagram



3.1 SocTop Architecture

STM32F103의 전체 구조를 참고하여 RV32I core, AXI-Lite/APB, AXI-Stream DMA, executable SRAM 기반으로 재설계한 SoC 본체

Core & Fast Path

- **Rv32Core:** Custom RV32I Core
- **IcodeLocalRom/SRAM:** ROM loader와 RAM app fetch 경로 분리

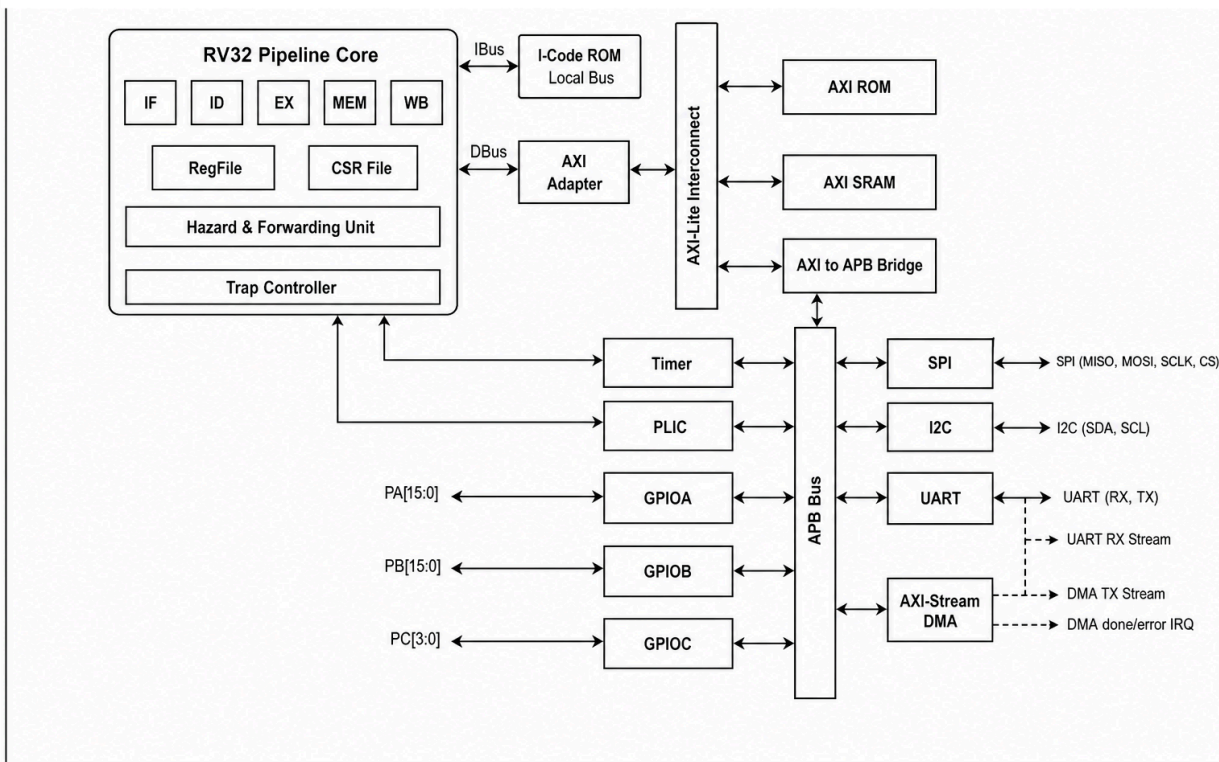
AXI-Lite Fabric & APB

- **AxiLiteInterconnect1x3:** AXI ROM window, SRAM, APB bridge 라우팅
- **PLIC-lite:** Interrupt 제어

Payload DMA

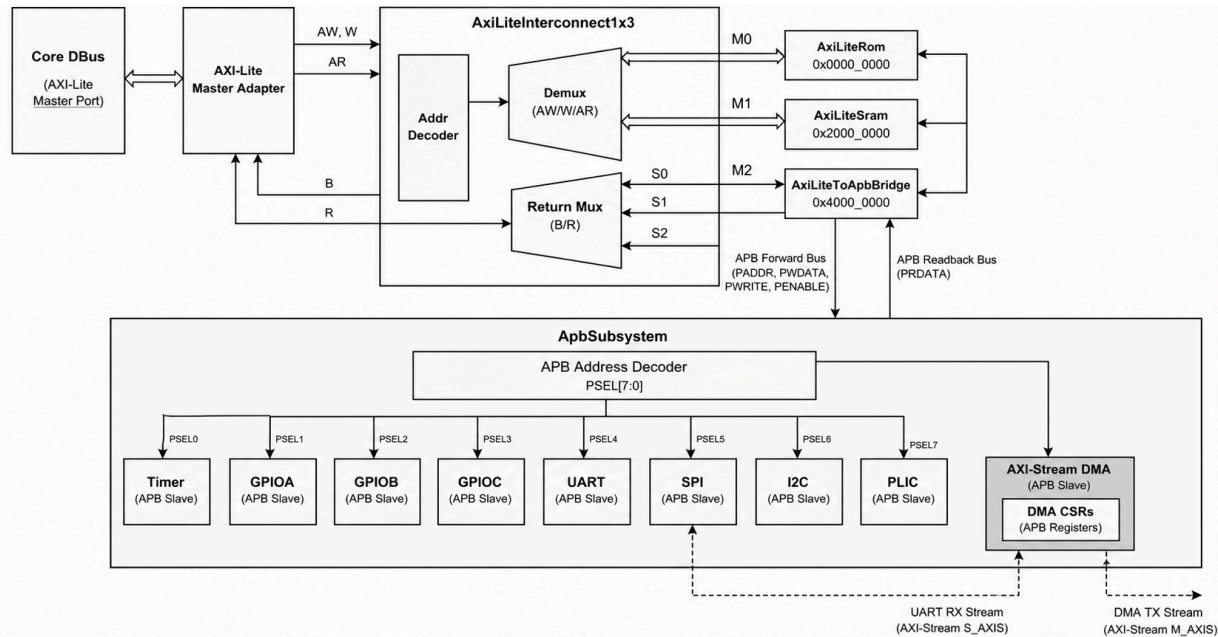
- **ApbAxiStreamDma:** CPU 개입 없이 대용량 데이터 전송
- **UART Loader:** C app을 SRAM에 적재 후 entry jump

Figure 2. RISC-V AXI SoC performance line block diagram



3.2 AXI-Lite Fabric & APB Bridge

Figure 3. Detailed Bus & Bridge Internal Architecture



단일 Master 구조

Core DBus → AXI-Lite Master Adapter를 거쳐 하나의 Master로 진입합니다.

AxiLiteInterconnect1x3

Address Decoder가 주소를 판별하고, Demux/Mux를 통해 ROM, SRAM, APB Bridge 3개의 Slave로 라우팅합니다.

APB Subsystem 통합

AxiLiteToApbBridge를 거친 PADDR, PWDATA 등의 신호는 다시 APB Address Decoder를 통해 각 Peripheral 칩셋을 구동합니다.

3.3 FPGA Top & Memory Map

FPGA Top Wrapper

SocFpgaTop은 Board pin, Clock(100MHz), Reset 스위치 등을 정리하는 Wrapper 역할을 수행합니다.

실질적인 Core, Bus, Peripheral, DMA 구조는 내부 SocTop에서 통합됩니다.

Memory / APB Map

Block	Base 주소	역할
ROM Window	0x0000_0000	고정 UART bootloader (IBus fetch fast path)
SRAM	0x2000_0000	RAM app 실행, stack, debug scratch
APB Window	0x4000_0000	AXI-Lite to APB bridge
Timer	0x4000_0000	Timer / IRQ
GPIOA	0x4001_0000	Switch / LED
GPIOB	0x4002_0000	GPIO 확장
SPI	0x4003_0000	SPI Master
I2C	0x4004_0000	I2C Master
UART	0x4005_0000	PC Serial
DMA	0x4006_0000	AXI-Stream DMA
GPIOC	0x4007_0000	GPIO 확장
PLIC-lite	0x400F_0000	Interrupt 제어기

4.1 Firmware Evolution & UART Bootloader

Before: ROM Firmware

초기 RGB IRQ demo는 assembly firmware를 ROM image로 직접 넣는 방식이었습니다.

- .S → ELF/BIN/MEM
- 새 firmware마다 bitstream rebuild
- 검증은 가능하지만 반복 속도가 느림



Now: ROM Loader

FPGA에는 UART loader ROM만 고정하고, 실제 앱은 SRAM에 내려받습니다.

- uart_loader.mem 고정
- UART packet 수신/검증
- SRAM write 후 entry jump



Result: C RAM App

PC에서 C app을 다시 빌드하고 UART로 다운로드해 보드에서 바로 실행합니다.

- Freestanding RV32I C runtime
- linker_ram.ld 기반 주소 배치
- LED, UART echo, DMA IRQ app 확장

ROM responsibility

packet receive, checksum/range validation, SRAM load, jump

PC responsibility

GCC build, objcopy binary, packet wrapping, serial download

Developer impact

RTL bitstream은 고정하고 firmware 실험 루프를 짧게 만듦

4.2 Loader Packet & RAM App Layout

Build / Download Pipeline

C source와 startup code를 `riscv64-unknown-elf-gcc`로 빌드하고, raw binary를 loader packet으로 감싼 뒤 UART로 전송합니다.

Executable SRAM Window

RAM app은 0x20001000부터 배치되고 stack top은 0x20004000입니다. SRAM 앞쪽 4KB는 debug scratch와 여유 영역으로 둡니다.

Interrupt-ready Runtime

`startup_irq.S`가 `mtvec`을 trap entry로 설정하고, C handler가 PLIC claim ID 기반으로 source를 dispatch합니다.

RAXI Loader Packet

Field	Size	Role
magic	4 bytes	"RAXI" signature
load_addr	4 bytes	default 0x20001000
byte_count	4 bytes	payload length after word padding
entry_addr	4 bytes	first PC after loader jump
checksum	4 bytes	additive byte checksum over payload
payload	N bytes	raw RAM app binary

핵심 변화: hardware SoC를 바꾸지 않는 한, 보드에는 loader bitstream을 한 번 올리고 이후 앱은 UART로 교체합니다.

5.1 AXI-Lite vs AXI-Stream

AXI-Lite

역할	CPU가 peripheral register를 읽고 쓰는 control path
주소	존재함. 접근할 타겟 레지스터의 주소 공간 지정
신호	5채널 (AW, W, B, AR, R) 기반의 독립적 트랜잭션
적용	CPU의 DMA, UART, SPI, PLIC 레지스터 설정 및 상태 폴링

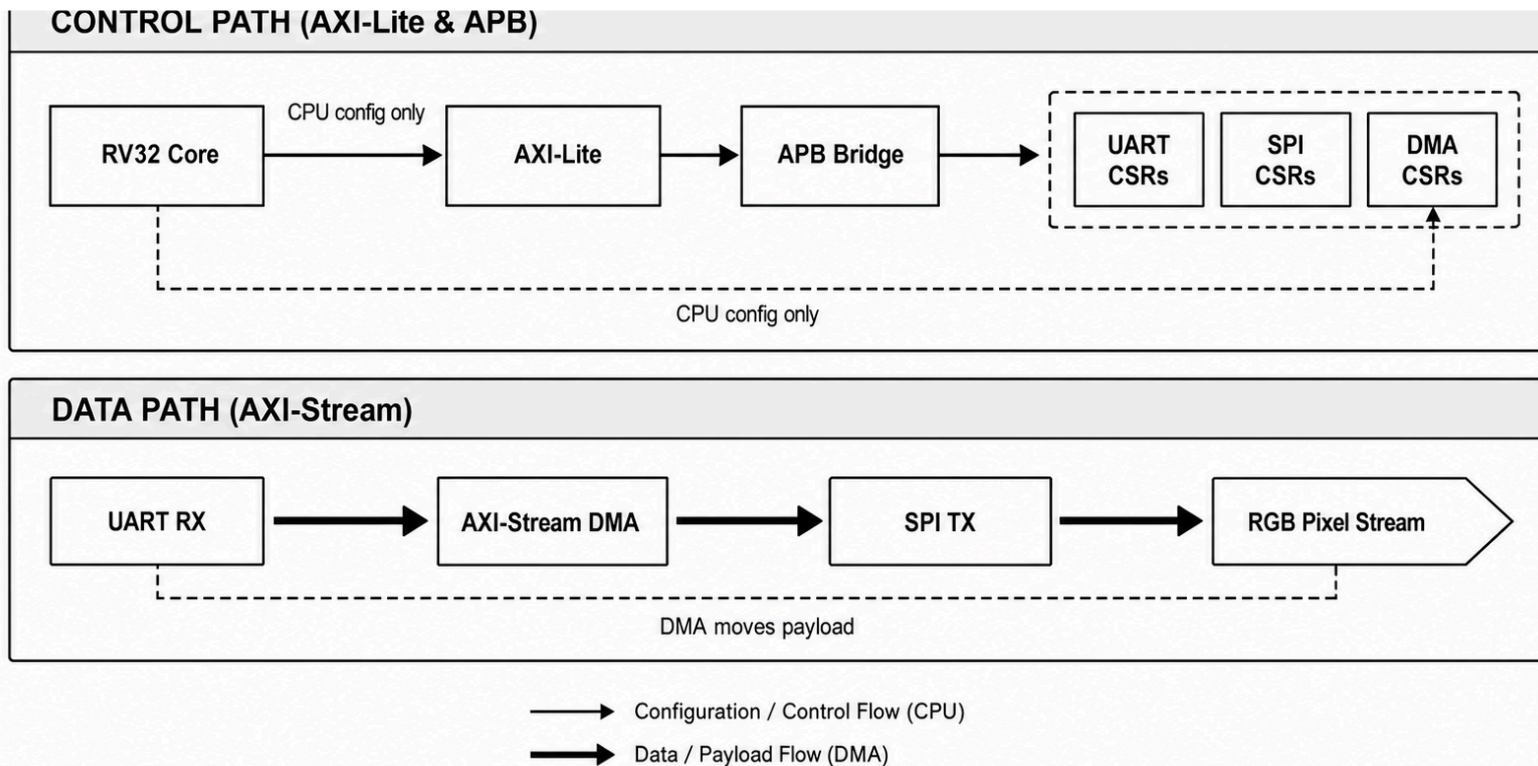
AXI-Stream

역할	byte payload가 흐르는 data path
주소	없음. Valid/Ready 핸드셰이크에 의한 순차적 데이터 흐름
신호	TVALID, TREADY, TDATA 중심의 단방향 전송
적용	UART RX → DMA 내부 버퍼 → SPI TX 간의 대규모 페이로드 이동

핵심 설계: AXI-Lite 기반 메모리 맵 제어와 AXI-Stream 기반 대용량 데이터 전송 경로의 분리

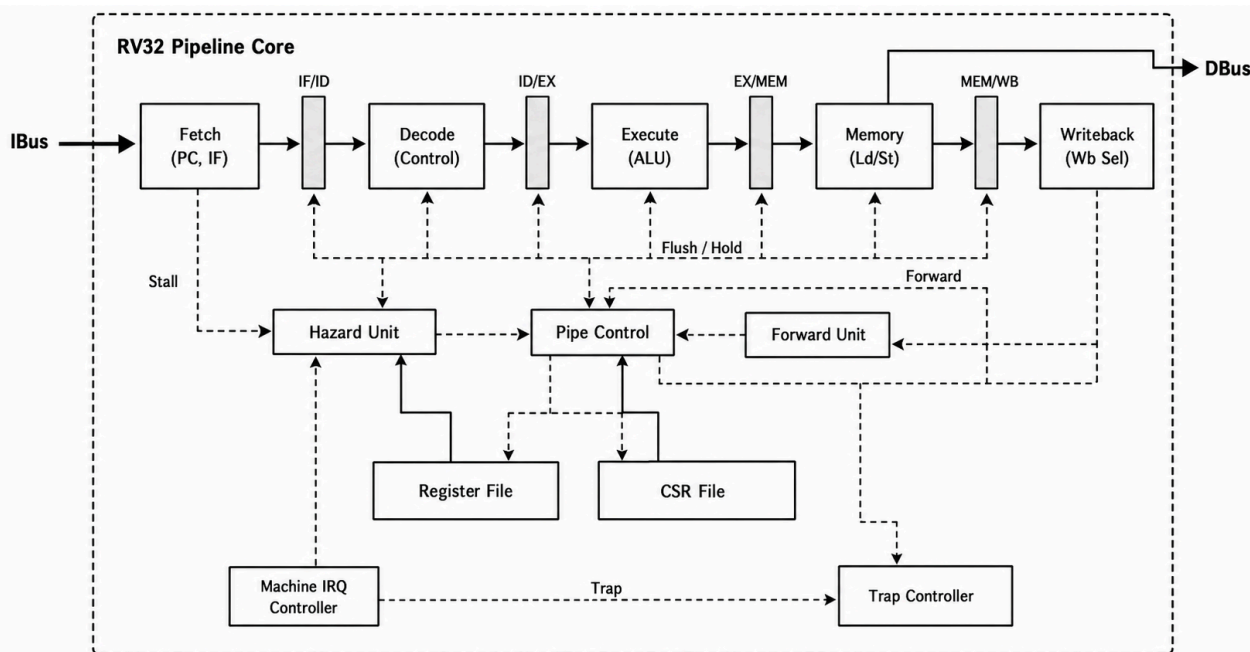
5.2 AXI Path Summary

Figure 4. System Data and Control Flow Separation



5.3 RV32 Pipeline Core

Figure 5. RV32I Pipeline Core Architecture



5-Stage Pipeline 구조

- **IF/ID/EX/MEM/WB:** 전형적인 RISC 5단계 파이프라인.
- **Forwarding Unit:** 데이터 해저드 완화를 위한 FWD A/B 바이패스 제공.

Trap & CSR

- **Trap Controller:** PLIC/Timer에서 온 외부 IRQ와 예외를 처리.
- **CSR File:** mepc, mcause 상태 기록 및 분기 제어.

Pipeline 제어 포인트

- **Hazard / Forward:** load-use, branch, data hazard를 stall/flush/forwarding으로 제어.
- **Fast Path:** IBus는 IcodeLocalRom으로 분리하고, payload 이동은 DMA로 offload.

5.4 Peripheral Architecture

APB Slave Interface

모든 주변장치는 표준화된 APB 노드를 통해 연결되며, 일관된 CSR(Control/Status Register) 인터페이스를 제공합니다.

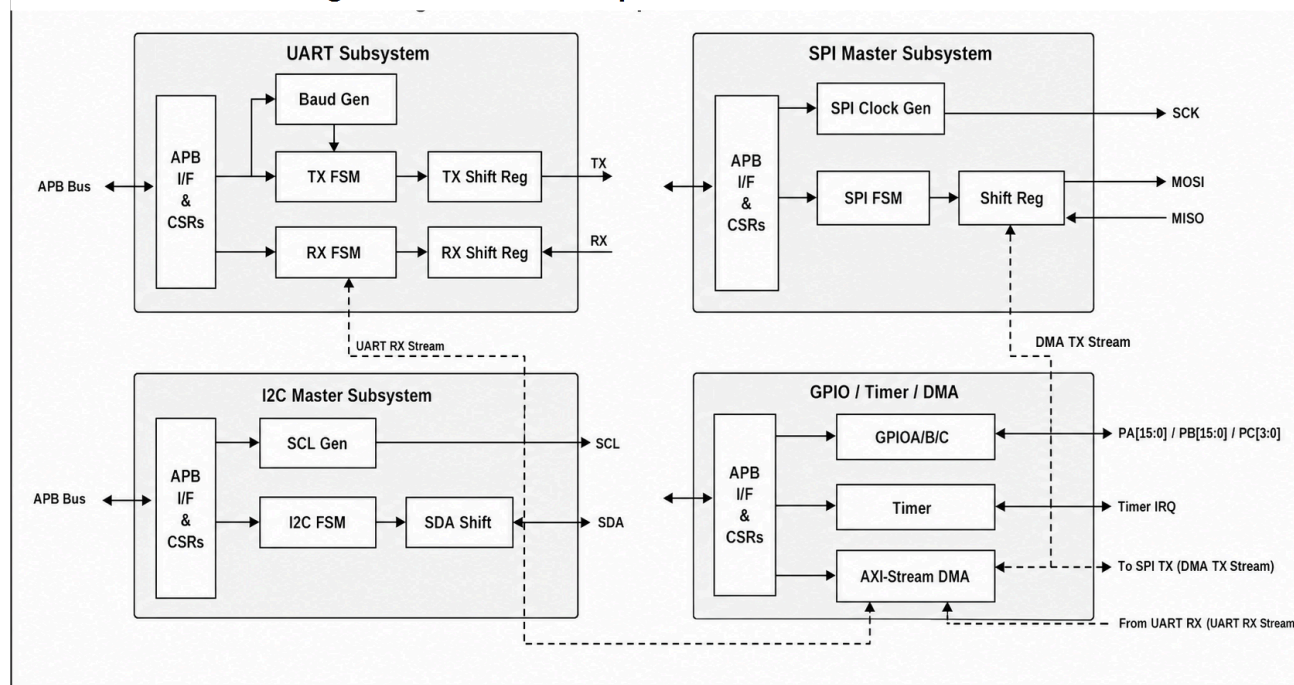
Peripheral 내부 블록

- **UART:** TX/RX Shift Register 및 오버샘플러
- **SPI:** SCK 생성 및 MOSI/MISO Shift 구동
- **I2C:** SCL 타이밍 및 Start/Stop/Ack 제어

IO 패드 매핑

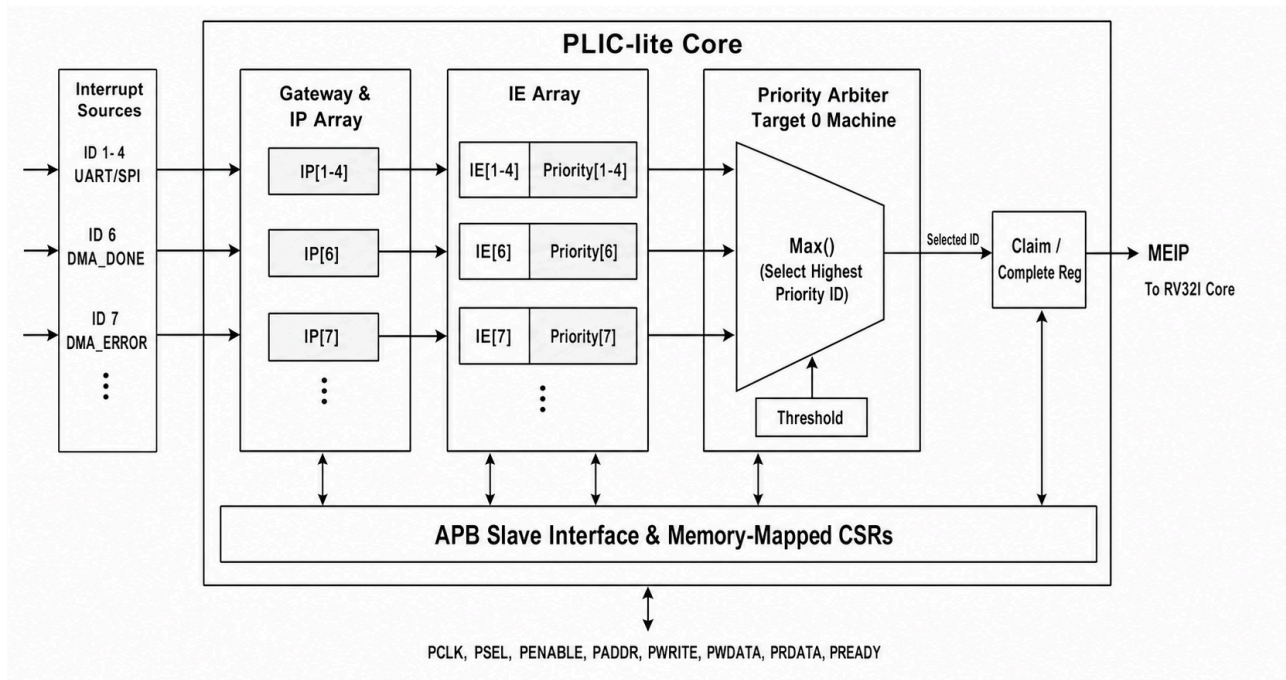
컨트롤러의 최종 입출력은 동기화 회로(Synchronizer)를 거쳐 물리적 IO Pad Buffer와 안전하게 연결됩니다.

Figure 6. Detailed Peripheral Internal Architecture



5.5 PLIC (Platform-Level Interrupt Controller)

Figure 7. PLIC Architecture



IRQ Source & Gateway

UART/SPI/DMA에서 발생한 raw IRQ는 Gateway를 거쳐 request pulse가 되고, complete 전까지 중복 요청을 막습니다.

Pending / Enable / Priority

pending source 중 enable되어 있고 threshold보다 priority가 높은 항목만 CPU가 claim할 수 있는 후보가 됩니다.

Arbiter & Claim/Complete

Arbiter가 priority가 가장 높은 source ID를 고르면 external IRQ가 올라갑니다. CPU는 CLAIM으로 ID를 읽고, 처리 후 COMPLETE에 같은 ID를 써서 정리합니다.

5.6 Interrupt Path & CSR

인터럽트 발생 구조

- 1 하드웨어 인터럽트 이벤트 발생 (예: DMA 전송 완료)
- 2 PLIC에서 우선순위를 판별하여 Machine External Interrupt (MEIP) Assert
- 3 Trap Controller가 실행 흐름을 멈추고 CSR (mepc, mcause) 하드웨어 업데이트
- 4 프로그램 카운터(PC)가 RAM app의 Direct Trap Entry(0x20001080)로 진입

관여하는 핵심 CSR

CSR	역할 설명
mtvec	Trap entry base (RAM IRQ app은 0x20001080)
mie.MEIE	Machine external interrupt enable
mstatus.MIE	Global machine interrupt enable
mcause	Interrupt 여부와 원인 기록
mepc	Trap 발생 전 PC 저장

5.7 Software Vector Table (vs ARM NVIC)

최종 interrupt routine은 실행 가능한 code 영역에 있습니다. 차이는 ARM NVIC는 하드웨어가 vector table에서 handler 주소를 직접 읽어 PC를 옮기고, 본 설계는 `mtvec` direct 진입 후 software가 PLIC Claim ID로 handler를 고른다는 점입니다.

비교 항목	ARM NVIC	현재 RISC-V firmware 설계
Table Lookup 주체	Hardware	Software
Entry 방식	IRQ number → Vector entry 이동	<code>mtvec=trap_entry</code> Direct Trap으로 통합 진입
Source 구분	IRQ number	PLIC Claim ID
Handler 분기	Hardware가 Handler PC Load	Software가 Table Load 후 <code>jalr</code> 분기
Routine 위치	Flash/ROM의 ISR code	UART로 적재된 RAM app의 trap/handler code

5.8 PLIC-lite Claim/Complete Flow

1. Raw IRQ Sources

UART/SPI/DMA interrupt level 입력



2. Gateway

1-cycle request pulse 생성, complete 전 중복 request 차단



3. Pending / Enable / Priority

pending latch 후 enable, threshold, priority 조건으로 claim 후보 선택



4. External IRQ to Core

claim 후보 존재 시 oExternalIrq가 올라가고 core는 external interrupt trap 진입



5. Software Claim/Complete

handler가 CLAIM ID를 읽고, 처리 후 COMPLETE에 같은 ID를 write

PLIC의 역할

handler 주소 대신 CPU에게 **interrupt source ID**를 알려줍니다.

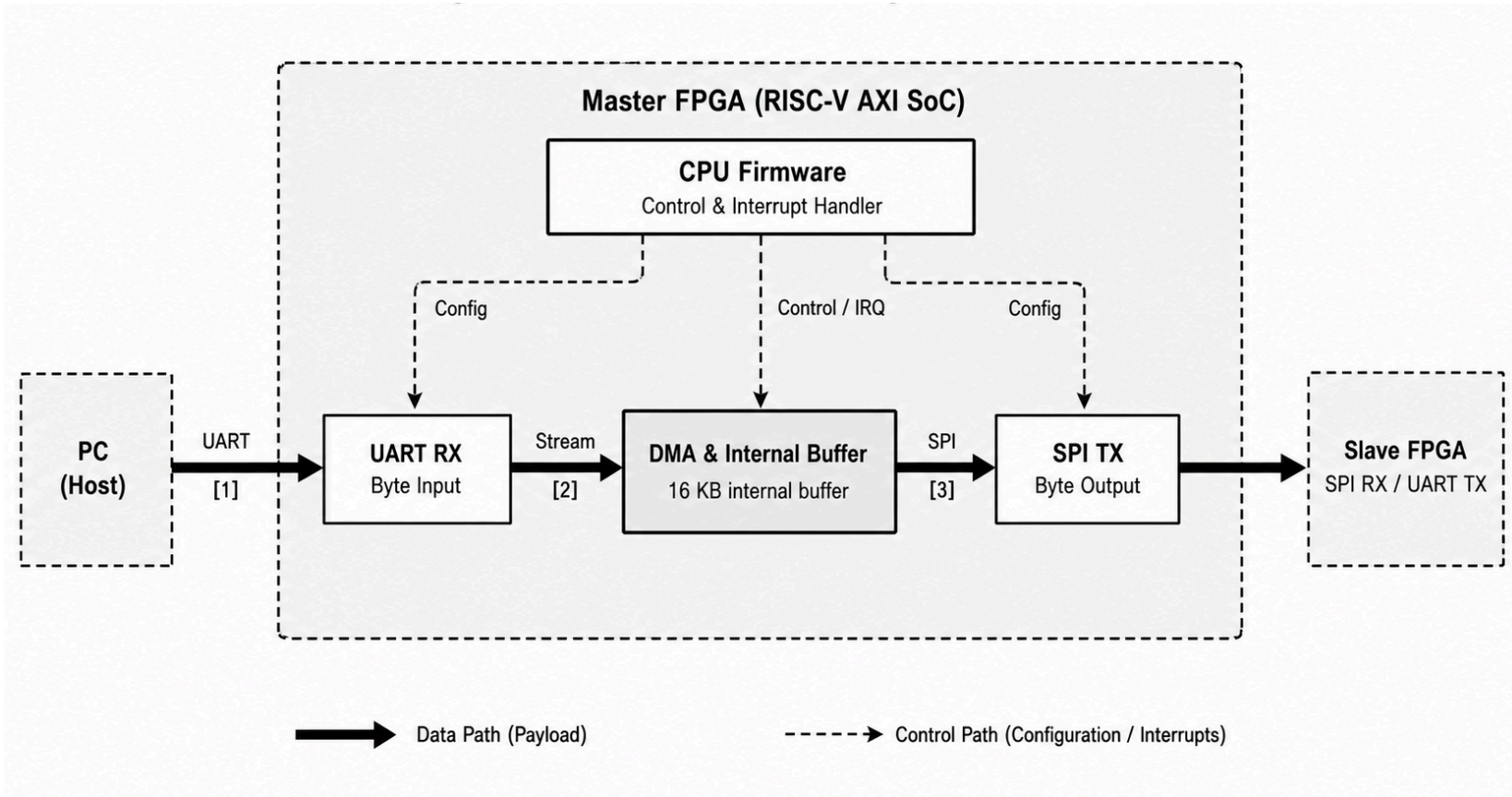
Source ID / Priority Map

ID	Source	Priority	현재 demo
1	UART_TX	1	미사용
2	UART_RX	1	미사용
3	SPI_TX	1	미사용
4	SPI_RX	1	미사용
5	Reserved / ext hook	1	미사용
6	DMA_DONE	1	handler 연결
7	DMA_ERROR	1	handler 연결
8	Reserved / ext hook	1	미사용

핵심: **priority** 값이 큰 source 먼저 claim, 동률은 낮은 ID. CLAIM은 ID 조회, COMPLETE는 처리 완료 정리.

6.1 End-to-End Image Transfer Scenario

Figure 8. End-to-End RGB Image Transfer Flow



6.2 AXI-Stream DMA in RAM App

Data Payload Movement

UART loader로 올라간 RAM app이 DMA register만 설정하고, RGB payload는 CPU register나 SRAM copy 없이 **DMA internal buffer**와 **Stream handshake**로 통신합니다.

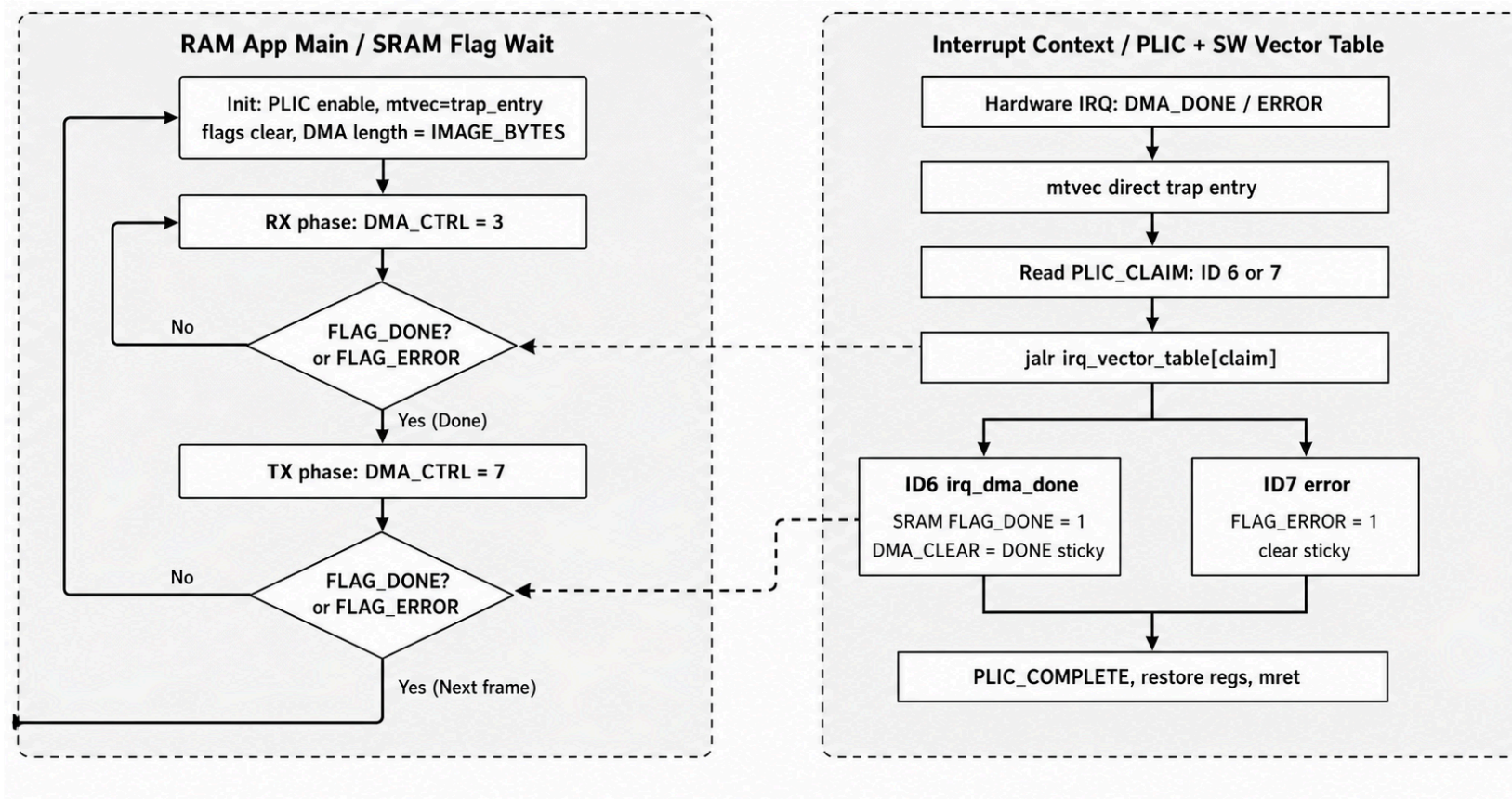
- UART RX Stream → DMA Internal Buffer (16 KB) → SPI TX Stream
- RGB888 64x64 프레임(12,288 bytes)을 한 번에 담을 수 있는 버퍼 크기.

RAM App의 역할 (DMA Register 설정)

Offset	Register	사용 예 (DMA stream scenario)
0x00	CTRL	3: RX start+IRQ 7: TX start+IRQ
0x08	LEN_BYTES	기본 전송 크기 12288
0x10	CLEAR	Done/error sticky clear
0x14	BUF_ADDR	Buffer offset 0

6.3 UART-to-SPI DMA IRQ Runtime Flow

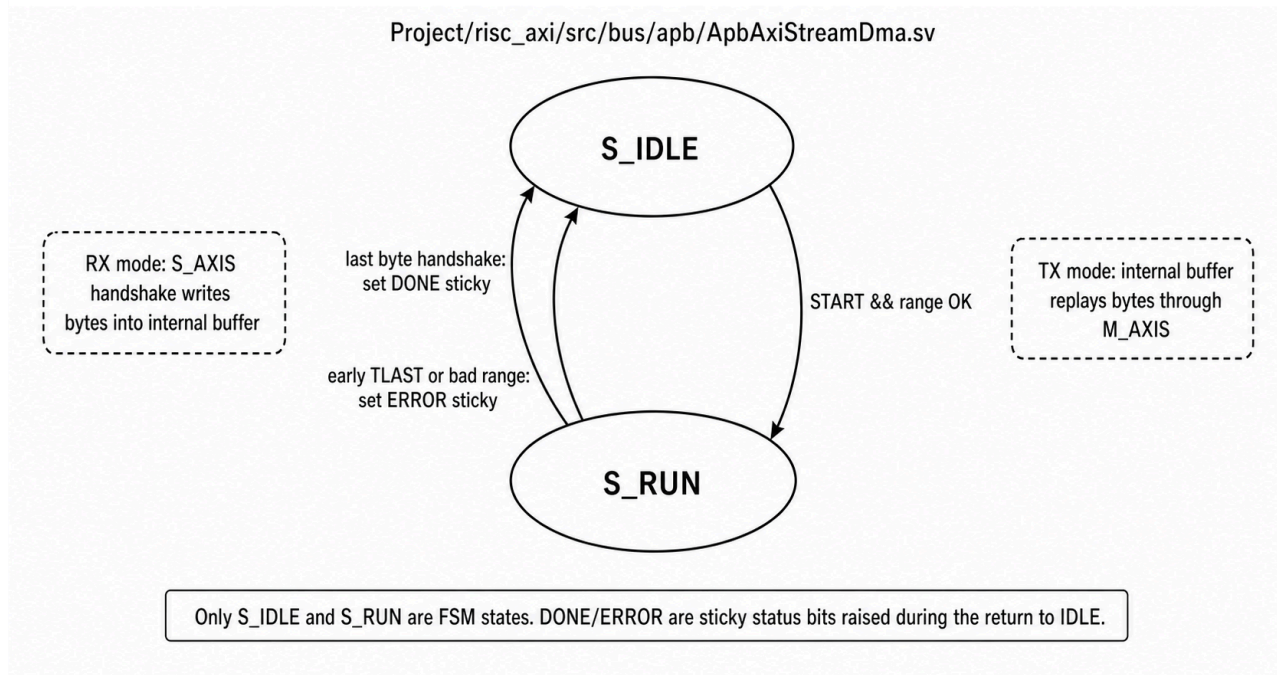
Figure 9. UART-to-SPI DMA RAM App Flowchart



7.1 Verification: DMA Stream Control FSM

Figure 10. APB AXI-Stream DMA FSM

Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv



Scenario

APB register 설정 후 DMA buffer의 pixel byte를 SPI TX stream으로 전달

Start

CPU가 DMA_CTRL=7을 쓰면 TX direction + IRQ enable로 실행

Handshake

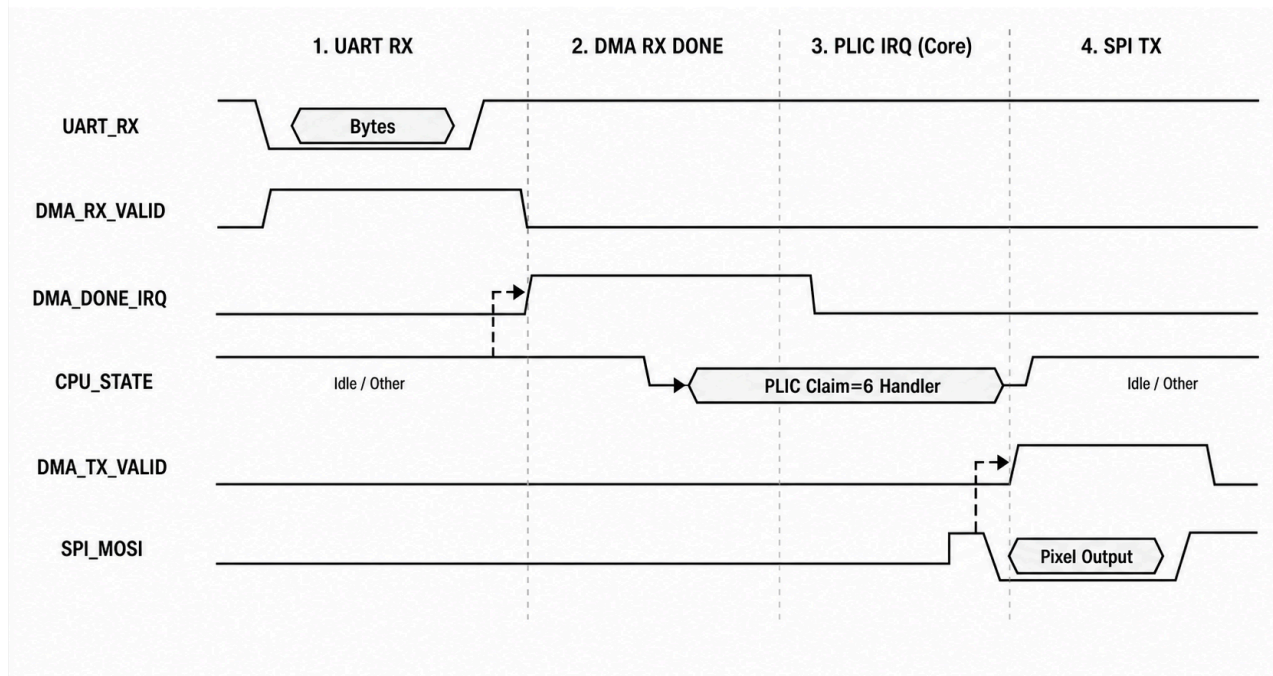
DMA M_TVAL ID/TDATA와 SPI TREADY가 byte 단위로 만남

Done IRQ

마지막 byte 이후 DMA_DONE sticky가 set되고 PLIC ID6으로 전달

7.2 TB Waveform Slot: SPI Pixel Pins

Figure 11. SPI Pixel Pin Waveform



Scenario

SPI TX가 받은 pixel byte를 외부 pin 파형으로 출력

Byte Frame

각 pixel byte가 SCK 구간에 맞춰 MOSI로 shift out

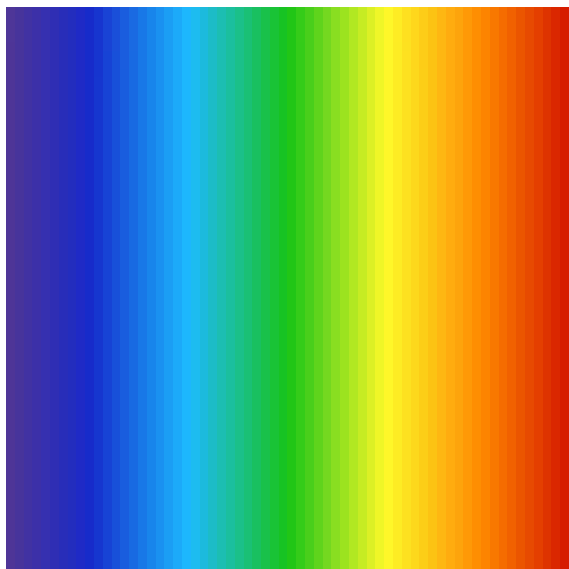
Observe

spi_sck, spi_mosi, spi_cs, TX busy/done 확인

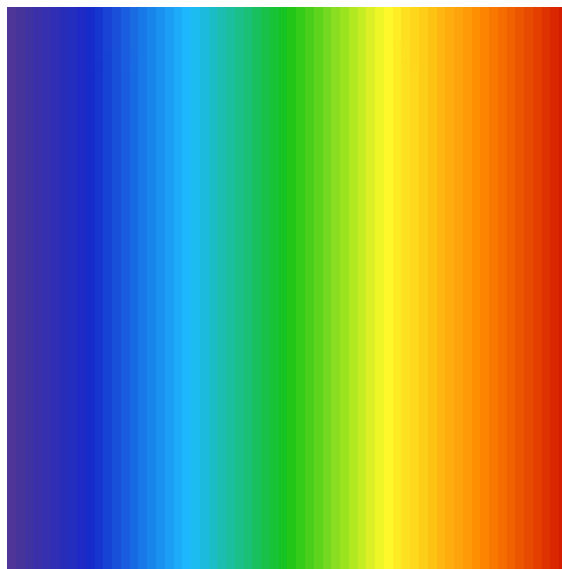
Check

expected pixel byte sequence와 MOSI 복원값 비교

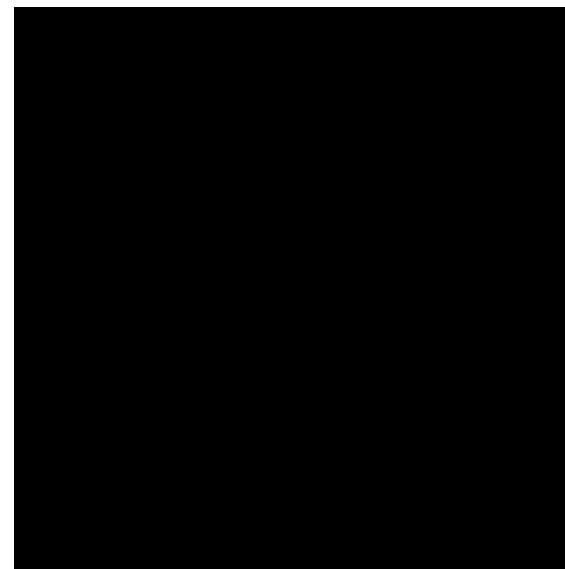
7.3 Demo: End-to-End Image Transfer



Original RGB888 64x64 image



Received image output



Diff / compare result

PC Sender

먼저 RAM app packet을 loader로 전송하고, 이후 RGB image bytes를 UART로 전송

Master FPGA

RAM app이 UART RX → DMA buffer
→ SPI TX 경로를 제어

Slave FPGA

SPI RX byte를 UART TX로 PC에 반환

Result Check

수신 이미지와 원본 이미지 비교

7.4 Result & Lessons Learned

Project Results

- AHB 기반 참고 구조에서 **AXI-Lite/APB/AXI-Stream** 구조로 성공적 전환
- RV32I core + AXI-Lite fabric + APB subsystem + AXI-Stream DMA 통합 구성
- UART bootloader로 C RAM app을 SRAM에 적재하고 0x20001000 entry 실행 확인
- PLIC claim ID 기반 Software dispatch 및 DMA IRQ phase 제어 증명

"처음 STM32F103 데이터시트를 봤을 때는 전체 구조가 잘 보이지 않아 막막했지만, core, bus, peripheral을 하나씩 공부하고 직접 구성하면서 데이터시트의 블록들이 어떤 역할로 연결되는지 조금씩 보이기 시작했습니다."

"Interrupt 처리에서 ARM NVIC처럼 hardware가 바로 vector table을 찾아가는 방식과, RISC-V에서 mtvec, **PLIC claim, software dispatch**를 직접 구성하는 방식의 근본적 차이를 체감했습니다."

"ROM에 assembly를 직접 올리던 방식에서 출발했지만, 이제는 **linker script, freestanding C runtime, UART loader packet**을 연결해 bitstream 재생성 없이 RAM app을 반복 실행할 수 있게 됐습니다."